SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Trabajo de consulta en parejas

Asignatura: Inteligencia de Negocios · Grupo 01

Integrantes:
Miguel Angel Pardo Alvarez
Jhon Herney Anacona Ortega

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Medellín 11/10/2026 - 2026 - 2

Propósito

En SolarBI el cuadro de mando final se construye en Power BI. Sin embargo, en la industria el monitoreo de plantas y equipos suele hacerse con Grafana. En esta consulta comparan las dos herramientas sobre los mismos datos y darán sus primeros pasos en tres temas que sostienen el proyecto: gobernanza de datos, automatización de procesos ETL e Internet de las Cosas (IoT), trabajando con control de versiones en GitHub.

Consulta

Respondan cada pregunta con sus propias palabras, en máximo 120 palabras, aplicándola al caso SolarBI e indicando la fuente. Las tablas, los diagramas y los fragmentos de código no cuentan en el límite.

Bloque 1 · Power BI y Grafana

- **¿Qué es Grafana?**

  Grafana es una plataforma de código abierto que permite visualizar, monitorear y analizar datos en tiempo real mediante paneles gráficos (dashboards). Se conecta a diferentes fuentes de datos para consultar métricas, registros y trazas, y permite configurar alertas que notifican cuando se presentan comportamientos inusuales o problemas en los sistemas. Es ampliamente utilizada para supervisar aplicaciones, servidores, bases de datos e infraestructura tecnológica.

  Describan sus componentes:

  - **Data sources:** conexión a PostgreSQL con las lecturas de los inversores
  - **Panels:** la unidad básica de visualización, un gráfico, un medidor o una tabla alimentada por una consulta.
  - **Dashboards:** tablero que agrupa varios paneles en una sola pantalla.
  - **Variables:** filtros dinámicos que cambian lo que muestran los paneles sin editarlos.
  - **Alerting**:reglas que evalúan una consulta cada cierto tiempo y envían una notificación si se cumple una condición
  - **Diferencien Grafana OSS de Grafana Cloud**: Grafana OSS es la versión gratuita y de código abierto, que cada usuario instala en su propio servidor y de la que es responsable de su mantenimiento. En cambio, Grafana Cloud es un servicio en la nube, con un plan gratuito limitado y planes de pago, donde Grafana Labs se encarga del servidor de cada usuario.

- **Elaboren una tabla comparativa Power BI vs. Grafana con mínimo 8 criterios**: Propósito principal, usuario típico, conexión a los datos, modelo semántico y lenguaje (DAX frente a la consulta del origen), actualización y tiempo real, alertas, licenciamiento y costo, y control de acceso.

Criterio	Power BI	Grafana
Propósito principal	Inteligencia de negocios y análisis histórico	Monitoreo y observabilidad casi en tiempo real
Usuario típico	Analista, gerencia, finanzas	Operador, ingeniero de O&M, TI

Conexión a los datos	Importación o Direct Query a cientos de orígenes	Consulta directa al origen mediante plugins (PostgreSQL, Prometheus
Modelo semántico	Modelo propio (tablas, relaciones, medidas)	Sin modelo; cada panel ejecuta su consulta
Lenguaje	DAX (medidas) y Power Query (M)	El del origen SQL, PromQL
Actualización y tiempo real	Programada (importación) o actualización automática de página; segundos solo con streaming	Refresco de segundos a minutos, nativo
Alertas	Alertas de datos sobre mosaicos del tablero en el servicio	Reglas de alerta con contact points y políticas de notificación
Licenciamiento y costo	Desktop gratis; servicio con licencias por usuario (Pro, PPU) o capacidad	OSS gratuito (AGPL); Cloud con plan gratuito y de pago
Control de acceso	Roles de workspace, RLS, Microsoft Entra ID	Organizaciones, equipos y roles (Viewer, Editor, Admin); RBAC en Enterprise y Cloud

- **Diferencien un dashboard operativo, uno analítico y uno estratégico**: Clasifiquen cada KPI mínimo del proyecto (energía, yield y Performance Ratio, ahorro, disponibilidad y alarmas, porcentaje de datos válidos) e indiquen en cuál de las dos herramientas lo publicarían y por qué.

    - **Operativo:** vigila el estado actual en tiempo real y apoya decisiones inmediatas del operador.
    - **Analítico:** explora datos históricos para encontrar causas y comparar periodos.
    - **Estratégico:** muestra pocos KPIs de alto nivel para la gerencia, con tendencia mensual o anual.

KPI	Tipo	Herramienta	Por qué
Energía generada (kWh)	Operativo (acumulado en estratégico)	Grafana	Se monitorea minuto a minuto; el acumulado mensual puede replicarse en Power BI
Yield (kWh/kWp) y Performance Ratio	Analítico	Power BI	Requiere cálculos con DAX y comparar periodos, plantas y condiciones de irradiación
Ahorro económico	Estratégico	Power BI	Combina energía con tarifas y es de interés gerencial
Disponibilidad y alarmas	Operativo	Grafana	Exige reacción rápida y se integra con alertas

% de datos válidos	Operativo	Grafana	Detecta fallas de adquisición de datos casi al instante

- **Grafana con PostgreSQL**: ¿Cómo debe ser una consulta de series de tiempo y para qué sirven las macros $__timeFilter y $__timeGroup? Escriban la consulta de la potencia promedio por hora.

  Una consulta de series de tiempo debe devolver una columna de tipo fecha-hora, preferiblemente llamada time y ordenada ascendentemente, y una o más columnas numéricas.
  - **$__timeFilter:** genera la condición BETWEEN con el rango de tiempo seleccionado en el dashboard.
  - **$__timeGroup:** agrupa los registros en intervalos fijos, y su variante

```
SELECT
$__timeGroupAlias(ts, '1h'),
AVG(potencia_kw) AS "Potencia promedio (kW)"
FROM lecturas_inversor
WHERE $__timeFilter(ts)
GROUP BY 1
ORDER BY 1;
```

- **Alertas:** Comparen Grafana Alerting (alert rules y contact points) con las alertas de datos del servicio de Power BI. Propongan una regla de alerta útil para la planta y el canal por el que se notificará.

  - **Grafana Alerting usa alert rules,** que ejecutan una consulta, aplican una condición y esperan un periodo de evaluación antes de disparar. Las políticas de notificación envían la alerta a un **contact point** (correo, Slack, Telegram, Teams o webhook). Las alertas de datos de Power BI se configuran sobre mosaicos numéricos (tarjeta, KPI, medidor) de un dashboard, con un umbral por encima o por debajo. Se evalúan sólo cuando los datos se actualizan y notifican por correo o por la app, o activan Power Automate.

  - **Regla de alerta propuesta:** si un inversor deja de enviar datos durante 10 minutos seguidos en horas de sol, Grafana dispara una alerta por falta de datos. Es útil porque, sin datos, no se sabe si la planta produce o si falló la medición. La notificación llegaría por correo electrónico al responsable de la planta y por Slack al equipo de soporte técnico.

Bloque 2 · Gobernanza de datos (data governance)

- **Definan data governance y diferenciarla de data management y de data quality:** Tomen como referencia el marco DAMA-DMBOK.

  - **Data governance** es el conjunto de reglas, roles y decisiones que definen quién manda sobre los datos y cómo deben usarse.
  - **Data management** es el trabajo diario de gestionarlos: almacenarlos, integrarlos, modelarlos y protegerlos.
  - **Data quality** mide si los datos sirven para su uso, según su exactitud, completitud, consistencia y oportunidad. La gobernanza fija las reglas, la gestión las ejecuta y la calidad comprueba que se cumplan.
  - **Según DAMA-DMBOK, data governance** es la autoridad que decide cómo se manejan los datos: define las políticas, los roles y las responsabilidades, y vigila que se cumplan. **Data management** es la ejecución: el

conjunto de actividades que almacenan, integran, modelan, protegen y entregan los datos durante todo su ciclo de vida. **Data quality** es una de las áreas de la gestión y mide qué tan aptos son los datos para su uso, según su exactitud, completitud, consistencia y oportunidad. En conclusión, la gobernanza decide las reglas, la gestión las aplica y la calidad comprueba si los datos las cumplen.

- **Expliquen los roles data owner, data steward y data custodian:** Relaciónalos con los roles del equipo en el proyecto (data engineer, data modeler, BI analyst y product owner).

  - El **data owner** es quien responde por un conjunto de datos y decide quién accede y qué calidad se exige
  - El **data steward** cuida el día a día: define qué significa cada campo, vigila su calidad y documenta.
  - El **data custodian** opera la infraestructura donde viven los datos: los almacena, respalda y protege.
  - **En SolarBI:** el owner sería el product owner, los stewards serían el BI analyst y el data modeler, y el custodian sería el data engineer.

- **El contrato de datos (data contract) del proyecto como instrumento de gobernanza:** ¿qué debe incluir (esquema, unidades, frecuencia, responsable, niveles de calidad)? ¿Qué le pasa al cuadro de mando si la fuente cambia un campo sin avisar?

  El contrato de datos es un acuerdo formal entre quien produce los datos y quien los consume. En SolarBI debe incluir el esquema (campos y tipos, como ts, potencia_kw y energia_kwh), las unidades (kW, kWh, °C), la frecuencia (una lectura cada 5 minutos y el retraso máximo aceptado), el responsable (owner y steward de cada campo) y los niveles de calidad (porcentaje mínimo de datos válidos, completitud y rangos permitidos). Si la fuente cambia un campo sin avisar, el ETL puede fallar o cargar nulos o valores en otra unidad, y el cuadro de mando mostraría KPIs vacíos o erróneos sin que nadie lo note. Con contrato, la validación en Silver detecta el cambio y genera una alerta

- **Seguridad a nivel de fila (row-level security, RLS) en Power BI:** roles y filtros DAX. Den el ejemplo de un responsable que solo ve su sitio. ¿Cuál es el mecanismo equivalente de control de acceso en Grafana (organizaciones, equipos y permisos por carpeta o dashboard)?

  - La RLS de Power BI limita qué filas ve cada usuario. Se crean **roles** en Power BI Desktop (Modelado > Administrar roles), cada uno con un **filtro DAX** sobre una tabla, y luego se asignan usuarios a esos roles en el servicio.
  - En Grafana no existe RLS dentro de una consulta. El control es por **organizaciones** (espacios separados), **equipos** y **permisos por carpeta o dashboard**, con roles Viewer, Editor y Admin. Se crearía una carpeta por sitio y solo el equipo de ese sitio tendría acceso a ella.

- **Ley 1581 de 2012 (protección de datos personales en Colombia):** ¿los datos de SolarBI son personales? ¿Qué cambiaría si el tablero mostrará qué operador atendió cada alarma? Fijen la postura del equipo frente al manejo responsable de la información.

  Los datos del inversor (potencia, energía, temperatura, estado) no son personales, porque no se refieren a una persona natural identificada o identificable (art. 3). Si el tablero mostrará qué operador atendió cada alarma, sí lo serían: identifican a alguien y revelan su desempeño laboral. Entonces se necesitaría autorización del titular, una finalidad definida, acceso restringido y medidas de seguridad. Postura del

equipo: recolectar solo lo necesario, registrar roles o códigos en lugar de nombres, restringir el acceso con carpetas en Grafana o RLS en Power BI, y no usar los datos para evaluar personas sin autorización.

Bloque 3 · Automatización de procesos ETL

- **Diferencien ETL de ELT y ubiquen cada paso en las capas Bronze, Silver y Gold del proyecto:** ¿Power Query es una herramienta ETL? Argumenten.

  - En **ETL** los datos se extraen, se transforman en una herramienta intermedia y solo después se cargan al destino.
  - En **ELT** se cargan primero tal como llegan y se transforman dentro del destino, por ejemplo con SQL en PostgreSQL.
  - Las capas encajan así: **Bronze** guarda los datos crudos sin cambios, **Silver** los limpia y valida (tipos, unidades, duplicados) y **Gold** los deja listos para el análisis (esquema estrella y KPIs).
  - **Power Query sí es una herramienta ETL**: Porque extrae mediante conectores, transforma con pasos en lenguaje M y carga al modelo. Pero es de autoservicio y se ejecuta dentro del refresco de Power BI, por lo que no reemplaza un ETL empresarial.

- **Comparen carga completa (full load) y carga incremental**. ¿Qué significa que una carga sea idempotente y cómo se logra en fact_energia_dia con su clave primaria (fecha_key, dispositivo_key) y un UPSERT?

  - En la **carga completa** se borra y se vuelve a cargar toda la tabla cada vez: es simple, pero se vuelve lenta cuando los datos crecen.
  - En la **carga incremental** solo se procesan los registros nuevos o modificados desde la última ejecución.
  - Una carga es **idempotente** si ejecutarla una o varias veces da el mismo resultado, sin duplicados; esto permite repetirla sin riesgo si falla a la mitad.
  - En **fact_energia_dia**, la clave primaria **fecha_key, dispositivo_key** garantiza una sola fila por día y equipo, y el UPSERT inserta las filas nuevas y actualiza las que ya existen.

- **Elaboren una tabla comparativa de tres formas de automatizar un flujo:** cron (o el Programador de tareas de Windows), Apache Airflow y la actualización programada de Power BI. Incluyan qué es, qué automatiza cada una y cuándo conviene.

	Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Programador del sistema operativo (cron en Linux, Programador de tareas en Windows)	Orquestador de código abierto en Python	Función del servicio de Power BI
Qué automatiza	Ejecutar un script o programa a una hora fija	Flujos con varias tareas (DAGs), dependencias, reintentos y monitoreo	El refresco del modelo semántico desde sus fuentes
Cuándo	Tareas simples e	Pipelines con varios	Mantener actualizado el cuadro

conviene	independientes, como lanzar un script ETL cada noche	pasos que dependen entre sí y necesitan control de fallas	de mando sin intervenir el ETL
Limitaciones	No maneja dependencias ni reintentos, y casi no tiene monitoreo	Requiere instalación y mantenimiento	Solo refresca el modelo; en Pro tiene un máximo de 8 actualizaciones diarias

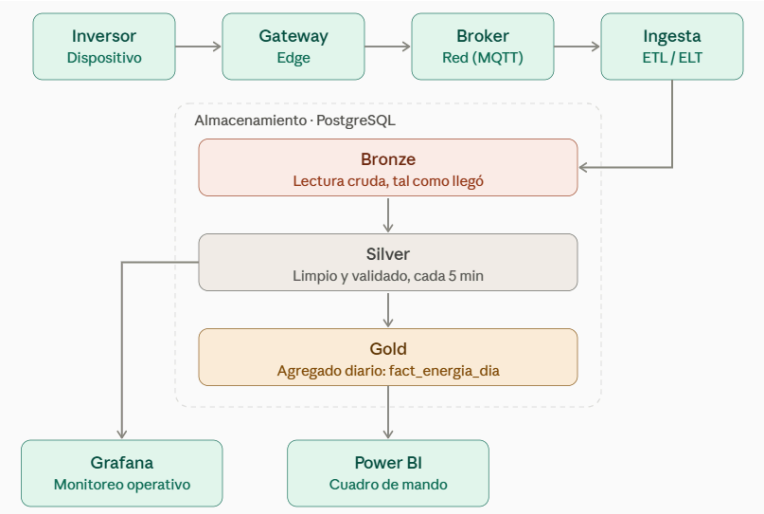
- **Query folding en Power Query**: ¿qué es, por qué mejora el rendimiento y cómo se verifica si un paso se está plegando hacia PostgreSQL?

  El query folding es la capacidad de Power Query de traducir los pasos de transformación a una única consulta nativa (en este caso SQL) que ejecuta PostgreSQL, en lugar de procesarlos en Power BI. Mejora el rendimiento porque filtros y agrupaciones se resuelven en la base de datos, cerca de los datos y aprovechando sus índices, y solo viaja a Power BI el resultado ya reducido. Para verificarlo, se hace clic derecho sobre un paso en el Editor de Power Query y se revisa la opción **Ver consulta nativa**: si está habilitada, el paso se pliega; si aparece deshabilitada, ese paso (o uno anterior) rompió el plegado

Bloque 4 · Internet de las Cosas (IoT)

- **Describan la arquitectura IoT por capas**: Dispositivo, gateway (edge), red o broker, almacenamiento y aplicación. Dibujen cómo viaja una lectura del inversor hasta llegar a un visual del tablero, indicando en qué capa (Bronze, Silver, Gold) queda en cada momento.

  - La arquitectura IoT se organiza en cinco capas. El **dispositivo** (inversor y sensores) mide potencia, energía y temperatura.
  - El **gateway o edge** lee esos datos por el protocolo del equipo (por ejemplo, Modbus), los ordena y los envía, y puede guardarlos si se cae la red.
  - La **red o broker** transporta los mensajes, por ejemplo un broker MQTT.
  - El **almacenamiento** guarda las lecturas en una base de datos como PostgreSQL.
  - La **aplicación** las muestra en Grafana o Power BI. La lectura llega primero cruda a Bronze, se limpia y válida en Silver y se agrega en Gold, de donde la toma el visual de Power BI.



- **Expliquen MQTT**: modelo publish/subscribe, broker, topic y niveles de QoS (0, 1 y 2). Propongan la jerarquía de topics de la planta.

    - MQTT es un protocolo ligero de mensajería para IoT. Funciona con **publish/subscribe**: los dispositivos publican mensajes y las aplicaciones se suscriben a los que les interesan, sin conocerse entre sí.
    - El **broker** recibe cada mensaje y lo reenvía a los suscriptores.
    - El **topic** es el nombre jerárquico, separado por /, que identifica de qué trata el mensaje.
    - El **QoS** define la garantía de entrega: **0**, como máximo una vez (puede perderse); **1**, al menos una vez (puede duplicarse); **2**, exactamente una vez (más lento). Para telemetría conviene QoS 1, porque los duplicados se absorben con una carga idempotente.

        - pascualbravo/solar/sitio01/inversor01/temperatura_c
        - pascualbravo/solar/sitio01/inversor01/estado

- **Describan una pila abierta típica de monitoreo IoT**: Mosquitto, Telegraf, una base de series de tiempo (InfluxDB o TimescaleDB) y Grafana. ¿Qué papel cumple cada componente y dónde encajaría PostgreSQL?

    - En esta pila, **Mosquitto** es el broker MQTT que recibe los mensajes de los equipos.
    - **Telegraf** es un agente que se suscribe al broker, toma esos mensajes y los escribe en la base de datos.
    - La **base de series de tiempo** (InfluxDB o TimescaleDB) guarda las lecturas con su marca de tiempo y permite consultarlas y agregarlas con eficiencia. **Grafana** las visualiza y genera alertas.
    - **PostgreSQL** encajaría como almacenamiento relacional del proyecto: dimensiones, tablas de hechos (Silver y Gold) y catálogo de dispositivos. Además, con la extensión TimescaleDB, el mismo PostgreSQL puede hacer de base de series de tiempo.

- **Procesamiento por lotes (batch) frente a flujo continuo (streaming)**. La telemetría llega cada 5 minutos y fact_energia_dia tiene granularidad diaria: ¿qué se gana y qué se pierde al agregar? ¿Qué herramienta atiende mejor cada granularidad?

    - El **batch** procesa los datos acumulados en lotes programados, como un ETL nocturno.
    - El **streaming** los procesa de forma continua, a medida que llegan. Con telemetría cada 5 minutos hay hasta 288 lecturas al día por variable y equipo, y **fact_energia_dia** las reduce a una fila.
    - Al agregar se **gana** menos volumen, consultas más rápidas y KPIs diarios estables.
    - Se **pierde** el detalle intradía: picos, caídas de potencia y la hora exacta de una falla, y ese detalle no se recupera desde el dato agregado. Por eso se conserva en Silver. Grafana atiende mejor la granularidad de 5 minutos (monitoreo operativo) y Power BI la diaria (análisis y gerencia).

Bloque 5 · Control de versiones

- **Diferencien Git de GitHub y definan repository, commit, branch y pull request. Un archivo .pbix es binario:** ¿Qué problema genera en Git y qué resuelve el formato Power BI Project (.pbip)? ¿Cómo se versiona un dashboard de Grafana?

    Git es el sistema de control de versiones que registra los cambios de un proyecto; GitHub es la plataforma en la nube que aloja repositorios Git y añade colaboración (pull requests y revisiones).

    - **Repository:** Carpeta del proyecto que guarda todos los archivos junto con su historial de cambios

- **Commit:** Registro de un conjunto de cambios, con un mensaje que explica qué se hizo
- **Branch:** Línea paralela de trabajo para probar cambios sin afectar la versión principal
- **Pull Request:** Solicitud en GitHub para revisar una rama y fusionarla con la principal

Un archivo .**pbix** es binario y comprimido: Git no puede mostrar qué cambió dentro ni fusionar dos versiones, así que cada edición reemplaza el archivo completo, el repositorio crece rápido y pueden colarse datos importados.

- El formato **.pbip** guarda el proyecto como una carpeta de archivos de texto (modelo semántico e informe), lo que permite ver diferencias, fusionar ramas y revisar cambios.
- Un dashboard de Grafana es un JSON: se exporta, se guarda en el repositorio y puede cargarse por provisioning.

Práctica guiada: los mismos datos en dos herramientas

Construirán un flujo pequeño pero completo: datos crudos de un dispositivo simulado, limpieza con reglas de calidad, carga en PostgreSQL y dos visualizaciones sobre esa misma base, una en Power BI y otra en Grafana.

Paso 1 · Bronze

datos crudos, Usen el dataset del contrato de datos del curso o, si lo prefieren, este simulador base (etl/simulador.py), que genera 3 días de telemetría de un inversor de 5 kWp con anomalías inyectadas. El archivo de Bronze no se modifica.

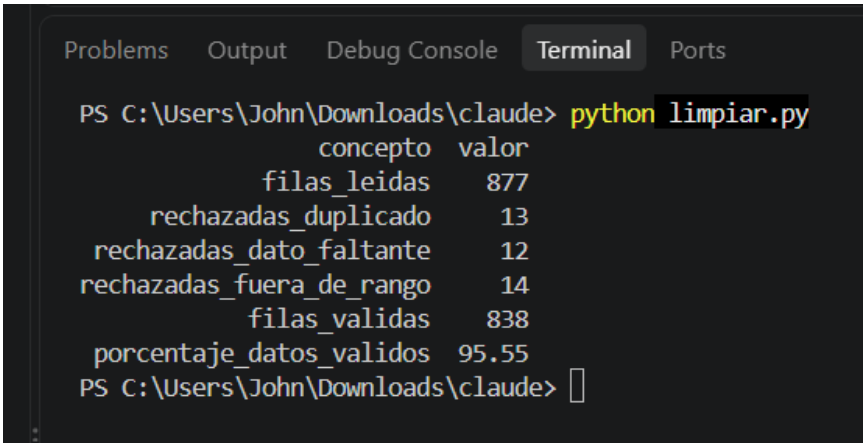
```
import csv, math, os, random
from datetime import datetime, timedelta

inicio = datetime(2026, 10, 5, 0, 0)
os.makedirs("data/bronze", exist_ok=True)

with open("data/bronze/telemetria.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
    for i in range(3 * 288):                    # 3 dias, cada 5 min
        ts = inicio + timedelta(minutes=5 * i)
        sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
        irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
        p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
        fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
        r = random.random()
        if r < 0.02:
            fila[2] = -1                        # anomalia: potencia negativa
        elif r < 0.04:
            fila[3] = ""                        # anomalia: dato faltante
        w.writerow(fila)
        if r > 0.98:
            w.writerow(fila)                    # anomalia: fila duplicada
```

Paso 2 · Silver: reglas de calidad

Con Python (pandas) o con SQL, apliquen mínimo tres reglas de calidad: rango físico válido, duplicados y datos faltantes. Reporten filas leídas, filas rechazadas por cada regla, filas válidas y el porcentaje de datos válidos.



- **Paso 3 · Gold: carga en PostgreSQL**
  - Carguen las lecturas limpias de 5 minutos en una tabla silver.lectura_5min.

Data Output | Messages | Notifications

Showing rows: 1 to 8

	ts [PK] timestamp without time zone	dispositivo_id [PK] integer	p_ac_kw numeric (8,3)	irradiancia_wm2 numeric (8,1)	temp_modulo_c numeric (5,1)
1	2026-10-05 06:05:00	1	0.080	18.1	22.7
2	2026-10-05 06:10:00	1	0.177	42.9	23.3
3	2026-10-05 06:15:00	1	0.201	47.2	24.0
4	2026-10-05 06:20:00	1	0.363	85.1	24.6
5	2026-10-05 06:25:00	1	0.381	90.1	25.3
6	2026-10-05 06:30:00	1	0.490	110.2	25.9
7	2026-10-05 06:35:00	1	0.601	145.1	26.6
8	2026-10-05 06:40:00	1	0.670	163.2	27.2

  - Carguen el resumen diario en una tabla con la estructura de dwh.fact_energia_dia (como mínimo: fecha, dispositivo, energia_kwh y pct_datos_validos). Recuerden que la energía de cada intervalo es la potencia por 5/60 de hora.

Data Output | Messages | Notifications

	fecha [PK] date	dispositivo_id [PK] integer	energia_kwh numeric (10,3)	pct_datos_validos numeric (5,2)
1	2026-10-05	1	26.580	95.88
2	2026-10-06	1	26.767	95.55
3	2026-10-07	1	26.699	95.24

- Dejen todo en un solo punto de entrada (etl/run_etl.py) que se ejecute con un único comando. Ejecútenlo dos veces y demuestren que los conteos no cambian (carga idempotente).

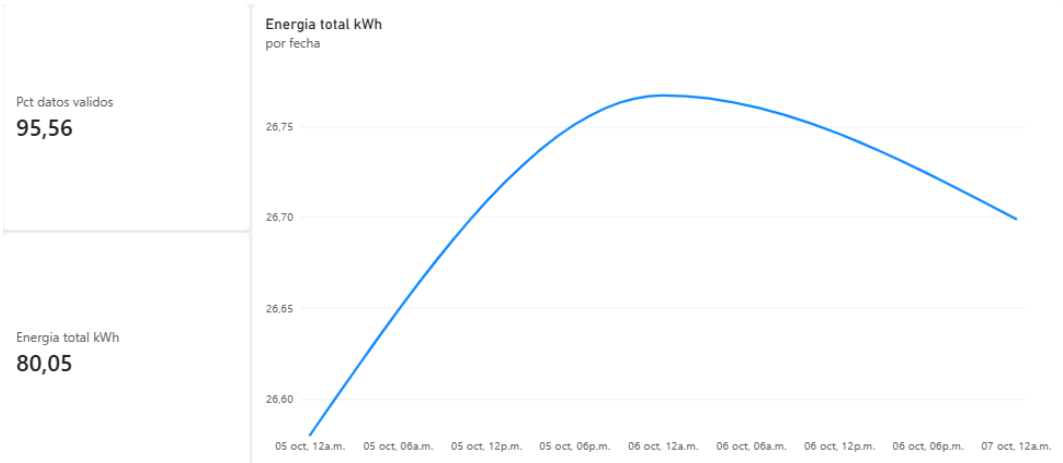
```
PS C:\Users\John\Downloads\claude> python run_etl.py
Contrasena de PostgreSQL (usuario postgres):
Calidad: {'leidas': 877, 'duplicado': 13, 'dato_faltante': 12, 'fuera_de_rango': 14, 'validas': 838, 'pct_validos': 95.55}
Filas en silver.lectura_5min:  838
Filas en dwh.fact_energia_dia: 3
PS C:\Users\John\Downloads\claude> python run_etl.py
Contrasena de PostgreSQL (usuario postgres):
Calidad: {'leidas': 877, 'duplicado': 13, 'dato_faltante': 12, 'fuera_de_rango': 14, 'validas': 838, 'pct_validos': 95.55}
Filas en silver.lectura_5min:  838
Filas en dwh.fact_energia_dia: 3
```

- Escriban, sin necesidad de implementarla, la expresión cron (o la configuración del Programador de tareas) que lanzaría ese comando cada día a la medianoche.

  - Este seria la expresion que lanzará utilizando cron **0 0 * * * cd /ruta/al/proyecto && python etl/run_etl.py**
  - **0** (primer campo, minuto): se ejecuta en el minuto 0 de la hora.
  - **0** (segundo campo, hora): se ejecuta a la hora 0, o sea, la medianoche.
  - ***** (día del mes): todos los días del mes.
  - ***** (mes): todos los meses del año.
  - ***** (día de la semana): todos los días de la semana.
  - **cd /ruta/al/proyecto:** entra a la carpeta del proyecto, porque el script busca data/bronze/ desde ahí.
  - **&&**: ejecuta el siguiente comando solo si el anterior funcionó.
  - **python etl/run_etl.py:** lanza el ETL completo: lee Bronze, aplica las reglas de calidad, carga Silver y actualiza el resumen diario.

- Paso 4 · Power BI

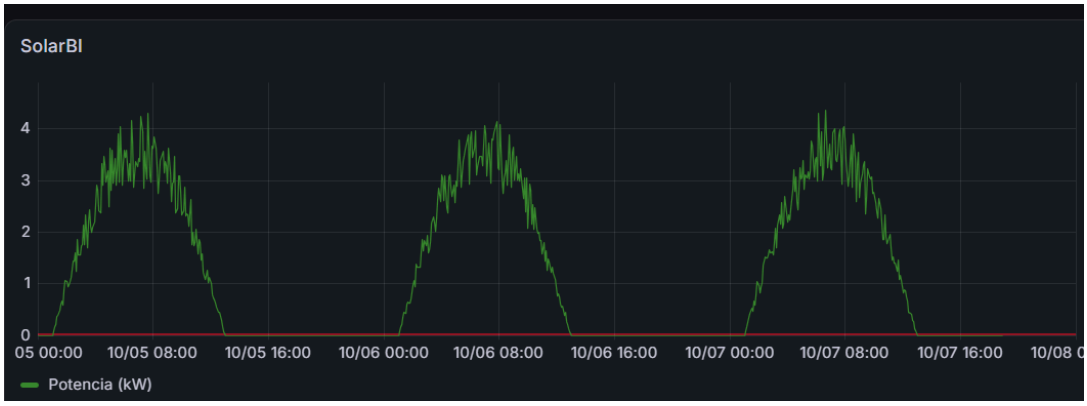
Conecten Power BI Desktop a PostgreSQL y construyan una página con: una tarjeta con la energía total (kWh), un gráfico de líneas con la energía diaria y una tarjeta con el porcentaje de datos válidos. Las cifras deben salir de medidas DAX, no de columnas arrastradas.



- **Paso 5 · Grafana**

Instalen Grafana OSS en su equipo (instalador o Docker), agreguen PostgreSQL como data source y construyan un dashboard con

- Un panel de series de tiempo con la potencia p_ac_kw cada 5 minutos, usando $__timeFilter en la consulta.



- **Paso 6 · Reflexión**

En máximo 10 líneas: con los mismos datos, ¿qué resolvió mejor cada herramienta?, ¿qué les costó más?, ¿cuál usarían para el operador de la planta y cuál para un directivo?

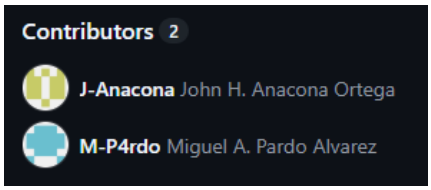
Al realizar cada prueba con los diferents herramientas nos dimos cuenta que power BI resolvió mejor en analisis historico de energia por dia, comparaciones y tambien un reporte claro y conciso, mientras que Grafana resolvió mejor el monitoreo operativo, con potencia casi en tiempo real, umbrales de color y una alerta de cuadno la potencia quedara en cero entre las 9:00 y las 15:00 que es lo que se habia pedido en el enunciado. Lo que mas nos costo en grafana fue la configuración, los colores de los umbrales funcionaban al revés de lo esperado y por otro lado la regla de alertar tiene varios pasos que son confusos para alguien que no había explorado la herramienta.Para el operador de la planta usaríamos Grafana, porque necesita ver en vivo si el inversor produce y recibir avisos cuando falla, para un directivo usaríamos Power BI, porque le interesan los totales, las tendencias y los indicadores resumidos.

Parte C · Repositorio en GitHub

Todo el trabajo se desarrolla y se entrega integrado a un repositorio en GitHub. Este repositorio no es de un solo uso: será la base sobre la que continuarán los siguientes trabajos del proyecto hasta el final del semestre.

Link: J-Anacona/SolarBI-Anacona-Alvarez.: Sistema escalable SolarBI

El repositorio quedó con un . en el nombre por ende se debe hacer el clone con:
git clone https://github.com/J-Anacona/SolarBI-Anacona-Alvarez..git SolarBI-Anacona-Alvarez



Referencias

JulCsc. (n.d.). Power BI documentation - Power BI. Microsoft Learn.
https://learn.microsoft.com/en-us/power-bi/

Grafana OSS and Enterprise | Grafana documentation. (n.d.). Grafana Labs.
https://grafana.com/docs/grafana/latest/

DAMA® Data Management Body of Knowledge (DAMA-DMBOK®) - DAMA International®. (2026, August 21). DAMA International®.
https://dama.org/learning-resources/dama-data-management-body-of-knowledge-dmbok/

Ley 1581 de 2012 - Gestor Normativo. (n.d.). Función Pública.
https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Concepto de broker y tópicos en MQTT. (n.d.).
https://cursos.asimov.mx/iot/modulo_3/concepto-de-broker-y-t%C3%B3picos-en-mqtt.html